

1. FastAPI – Introduction^{FastAPI –}

FastAPI is a modern Python web framework, very efficient in building APIs. It is based on Python's type hints feature that has been added since Python 3.6 onwards. It is one of the fastest web frameworks of Python.

- As it works on the functionality of Starlette and Pydantic libraries, its performance is amongst the best and on par with that of NodeJS and Go.
- In addition to offering high performance, FastAPI offers significant speed for development, reduces human-induced errors in the code, is easy to learn and is completely production-ready.
- FastAPI is fully compatible with well-known standards of APIs, namely OpenAPI and JSON schema.

FastAPI has been developed by **Sebastian Ramirez** in Dec. 2018. FastAPI 0.68.0 is the currently available version.

FastAPI – Environment Setup

To install FastAPI (preferably in a virtual environment), use **pip** installer.

```
pip3 install fastapi
```

FastAPI depends on **Starlette** and **Pydantic** libraries, hence they also get

Installing Uvicorn using PIP

FastAPI doesn't come with any built-in server application. To run FastAPI app, you need an ASGI server called **uvicorn**, so install the same too, using pip installer. It will also install uvicorn's dependencies - asgiref, click, h11, and typing-extensions

```
pip3 install uvicorn
```

With these two libraries installed, we can check all the libraries installed so far

Getting Started

The first step in creating a FastAPI app is to declare the application object of FastAPI class.

```
from fastapi import FastAPI app = FastAPI()
```

This **app** object is the main point of interaction of the application with the client browser. The uvicorn server uses this object to listen to client's request.

The next step is to create path operation. Path is a URL which when visited by the client invokes visits a mapped URL to one of the HTTP methods, an associated function is to be executed. We need to bind a view function to a URL and the corresponding HTTP method. For example, the **index()** function corresponds to **'/'** path with **'get'** operation.

```
@app.get("/")
async def root():
    return {"message": "Hello World"}
```

The function returns a JSON response, however, it can return **dict**, **list**, **str**, **int**, etc. It can also return Pydantic models.